

Desafio Flutter - App de Carrinho de Compras (Arquitetura MVVM)

Objetivo

Desenvolver um aplicativo Flutter com fluxo de compra utilizando a arquitetura MVVM recomendada pelo time do Flutter: camadas bem separadas, reatividade com `ChangeNotifier`, utilização dos padrões `Command/Result` e navegação com rotas nomeadas.

O que será avaliado

- Aderência à arquitetura MVVM (separação clara entre Model, View e ViewModel).
- Consistência no uso dos padrões Command/Result.
- Consistência nos padrões utilizados em cada camada da aplicação.
- Estrutura e clareza do código (incluindo, mas não se restringindo à organização de pastas).
- Uso adequado de widgets, navegação e gerenciamento de estado reativo.
- Componentização e separação de responsabilidades dos elementos de ui.
- UX consistente.
- Exibição das mensagens de erros para o usuário, retornados pelos serviços de api.
- Consumo de API (serviços, erros e estados de loading/exceptions no ViewModel).
- Lógica de carrinho (adição, remoção, atualização de quantidades, totalização).
- Fonte única de verdade e persistência global do carrinho.
- Aplicação correta das regras de negócio.
- Uso de Git (commits claros).
- Qualidade do README (setup, arquitetura, decisões).

Requisitos do Desafio

Carrinho

1. O carrinho deve ser um estado global sem dependências de serviços de api ou eventuais repositórios e use cases.
2. O estado do carrinho deve ser acessado pelas telas de catálogo e carrinho.
3. Regra de negócio:
 - a. Não deve ser possível a inclusão de mais de 10 produtos diferentes.
 - b. Não deve ser possível editar um carrinho já finalizado.
 - c. As regras de negócio devem ser definidas na camada mais apropriadas da aplicação
4. Todas as interações com o carrinho devem:
 - a. conter as chamadas de api simuladas (utilizar `Future.delayed`) no serviço de api.

- b. atualizar a store com os dados corretamente.

Navegação

5. Utilização de rotas nomeadas (navegação nativa do flutter, sem pacotes).
6. Não deve ser possível voltar a stack de navegação para a tela de carrinho após o checkout.

Tela inicial (Catálogo de produtos)

7. Lista de produtos com imagem, nome, preço unitário e botão “Adicionar ao carrinho”
8. Dados da API: 🖱️ <https://fakestoreapi.com/products>
9. Exibir loading e estados de erro expostos pelos **Commands** do **ViewModel** da tela.
10. Após a adição do produto no carrinho, o botão adicionar dá lugar a um contador que exibe a quantidade atual de itens do produto no carrinho. Esse contador exibe também as ações de incrementar e decrementar itens do produto no carrinho.
11. Ícone no AppBar com badge da quantidade total de itens diferentes no carrinho.

Tela de Carrinho

12. Listar itens com imagem, nome, quantidade de itens, preço unitário, subtotal e controles de quantidade (+/-).
13. Remover item.
14. Resumo do pedido: Quantidade de itens, subtotal, total.
15. Botão finalizar: Deve realizar uma chamada (simulada) para a api de checkout
 - a. Após resultado com sucesso deve navegar para a tela de pedido finalizado

Tela de pedido finalizado

16. Listar itens com imagem, nome, preço unitário, subtotal
17. Resumo do pedido: subtotal, frete (simulado), total.
18. Botão "Novo pedido": retorna para a tela de catálogo com um novo carrinho limpo.

🧩 Requisitos Técnicos (MVVM)

- Flutter 3.x
- MVVM com camadas e responsabilidades claras:
 - Model
 - Entidades e DTOs (ex.: **Product**, **Cart** e **CartItem**).
 - View
 - Renderiza a UI reagindo a estados dos **ViewModels** e da **CartStore**.
 - Escuta eventuais listeners de commands
 - ViewModel
 - Vinculado ao ciclo de vida das views
 - Lida com os estados *results* das demais camadas da aplicação e, quando necessário, repassa para a view.
 - Orquestra casos de uso e/ou chamadas dos serviços de api.

- Data (Service)
 - `ProductsApi` (<https://fakestoreapi.com/products>)
 - `CartApi` (simulação com delay)
 - `CheckoutApi` (simulação com delay)
 - Simular erro em todas as chamadas de api de forma randômica.
- Gerenciamento de estado: `ChangeNotifier`
- Tratamento apropriado de erros nas diferentes camadas da aplicação
- Persistência global para carrinho (apenas na memória).
- Código versionado no GitHub.
- README com instruções e diagrama/resumo de arquitetura.

✨ Extras (diferenciais)

- Testes unitários para os módulos da camada de domínio.
- Testes unitários de ViewModels (ex.: fluxo de loading/erro).
- Testes de integração das telas simulando a execução do ponto de vista de um usuário.
- Demonstrar o uso de Clean Architecture por meio de desacoplamento entre domínio, infraestrutura e apresentação, incluindo mas não se limitando à organização de pastas ou nomenclaturas.

🔌 Integração com API

- HTTP via `http` ou `dio`.
- Serialização em DTOs da camada de dados; entidades puras na domínio.

📦 Entrega

- Repositório público no GitHub chamado `flutter-shopping-cart-mvvm`.
- README com:
 - Requisitos (versão do Flutter/Dart).
 - Instruções de build/run.
 - Opcional: GIFs/screenshots (lista, detalhes, carrinho, checkout).
 - Commits por feature principais (produtos, carrinho, checkout), sem granularidade excessiva, salvo correções pontuais.

🔍 Observações

- Observe a clareza nas mensagens de commit.
- Separe corretamente as features implementadas em seus respectivos commits.
- Mantenha a separação de responsabilidades de cada camada da aplicação.
- Priorize clareza e simplicidade no código, tratamento de erros e boa UX.